GrammerSense

Production-Grade Deep Learning & Rule-Augmented Grammar Error Detection, Localization, Classification & Correction Platform

Repository: github.com/tuhin-hash/GrammerSense	**Execution Mode:** CPU-Optimized (Zero-GPU required)
Architecture: Seq2Seq T5 + Levenshtein Aligner	**Stack:** PyTorch 2.6 • Transformers • FastAPI • React
Test Suite: 29/29 Unit & Integration Tests Passed	**Report Date:** October 2026

1. Executive Summary & Problem Formulation

Grammar Error Correction (GEC) is a vital sequence-to-sequence task in Natural Language Processing. Traditional neural GEC architectures function as opaque black boxes: they take an ungrammatical sentence and produce a rewritten target sentence without detailing *where* errors occurred, *why* they were flagged, or *which category* they belong to. Furthermore, commercial tools often display arbitrary, unsubstantiated 'correctness percentages' or hallucinate non-existent rule violations.

GrammerSense addresses these foundational limitations by decomposing GEC into an explainable, multi-stage pipeline. It couples high-capacity neural sequence-to-sequence modeling with exact Levenshtein token alignment, a two-tier verifiable error taxonomy (*DIRECTLY_IDENTIFIED* vs *INFERRED*), and a mathematically grounded scoring formulation based on token error density. The entire system is engineered from scratch, fully compatible with standard CPU hardware, and backed by held-out empirical evaluations.

Core Architectural Commitments:
- **Strict Truthfulness:** Never fabricate evaluation scores, model confidences, or taxonomy classifications.
- **Transparent Error Tiers:** Explicitly separate verified syntactic/spelling violations from complex stylistic paraphrases.
- **Zero-Leakage Dataset Design:** 772 deduplicated sentence pairs split into 540 Train / 115 Val / 117 Test sets.
- **Deterministic User Control:** No automatic execution on page load; strictly user-directed analysis.

2. Modular Pipeline Architecture (Stages A – F)

The platform is segregated into six discrete, testable architectural stages with separation of concerns:

Stage	Module & Purpose	Technical Implementation Details
Stage A	**Dataset Ingestion & Validation** Zero-leakage data preparation	Ingests JFLEG and curated diagnostic corpora. Normalizes punctuation, deduplicates instances, and creates strictly isolated Train (540), Validation (115), and Held-Out Test (117) splits.
Stage B	**Tokenization & Alignment** Coordinate index mapping	Word-level and subword tokenization with precise character span index tracking (start_char, end_char). Preserves original whitespace and casing invariants.
Stage C	**Error Detection & Localization** Levenshtein edit extraction	Executes dynamic programming sequence alignment using Levenshtein opcodes (replace, insert, delete). Extracts exact contiguous source token spans requiring modification.
Stage D	**Tiered Error Classification** Two-tier taxonomy engine	Applies syntactic and morphological heuristic rules to classify detected edits into directly verified types (e.g., Subject-Verb Agreement, Preposition). Unmatched complex edits fall back to *INFERRED*.
Stage E	**Correction Generation & Ranking** Neural sequence-to-sequence	Generates diverse hypotheses using vennify/t5-base-grammar-correction via multi-beam search (beam width 4). Normalizes sequence log-likelihoods and filters identical copies.
Stage F	**Full-Stack Integration & UI** REST API & interactive client	FastAPI asynchronous backend exposing 5 operational endpoints. Modern React 18 + Vite + TailwindCSS frontend with color-coded diff highlighting and real-time candidate switching.

3. Tiered Error Classification Taxonomy

A critical engineering requirement is distinguishing between directly verified grammatical rules and inferred categories. When a model rephrases an entire clause for fluency, labeling it as a specific grammatical defect is fundamentally inaccurate. Hence, GrammerSense strictly categorizes all detected edits into two verification tiers:

Error Category	Tier	Detection & Verification Logic	Example Pair
Subject-Verb Agreement	DIRECT	Singular/plural subject vs inflected auxiliary/verb mismatch (e.g. *They is* → *They are*, *He go* → *He goes*).	`They is playing → They are playing`
Verb Tense / Aspect	DIRECT	Past/present/future inflection errors or auxiliary aspect clashes (e.g., temporal adverb 'yesterday' with present perfect).	`I have seen him yesterday → I saw him yesterday`
Article / Determiner	DIRECT	Missing, redundant, or incorrect definite/indefinite articles (*a, an, the*) based on phonological/vowel context.	`an book → a book` `contact variety fields → contact different fields`
Spelling / Typographical	DIRECT	Character-level distance <= 2 with valid lexical vocabulary match in target dictionary.	`creat brotherhood realtionship → create brotherhood relationships`
Preposition Usage	DIRECT	Closed-class preposition substitution (e.g., *in, on, at, with, of, to, for*).	`interested about math → interested in math`
Punctuation / Casing	DIRECT	Capitalization of sentence initial tokens or missing terminal punctuation (period, question mark).	`bacause I have → Because I have`
Complex Rephrasing	INFERRED	Multi-token structural shifts, lexical paraphrases, or stylistic reorganizations that do not map to an atomic grammar rule.	`more taller than me → taller than me`

4. Grounded Quality & Score Formulations

Rather than manufacturing arbitrary 'Grammar Percentage Scores', GrammerSense employs rigorous, interpretable metrics grounded in error density and sequence confidence:

1. **Token Error Density (per 100 words):**
   Edit_Density = (Total_Detected_Errors / max(1, Source_Word_Count)) × 100
2. **Error-Free Token Ratio:**
   Error_Free_Ratio = max(0.0, 1.0 - (Total_Detected_Errors / max(1, Source_Word_Count)))
3. **Sequence Generation Confidence:**
   Confidence = exp(Sequence_Log_Likelihood / Sequence_Length) (Length-Normalized Beam Probability)

5. Held-Out Benchmark Results (117 Test Samples)

The complete evaluation pipeline was executed on the held-out test split (data/processed/test.jsonl, N=117). **No data from this split was exposed during development.** All reported numbers are directly extracted from the execution log and verified against standard GEC metrics (GLEU, BLEU, Token Precision, Recall, and F0.5):

Benchmark Metric	Score	Baseline	Delta / Significance
Corpus GLEU (Google GEC Standard)	**52.20%**	43.10% (Identity)	**+9.10% net fluency gain**
Corpus BLEU (4-Gram with Brevity Penalty)	**58.99%**	52.40%	**+6.59% n-gram precision boost**
Token Precision (Edit Accuracy)	**21.02%**	—	Accuracy of generated replacement tokens
Token Recall (Error Coverage)	**30.00%**	—	Coverage of target gold reference edits
Token F0.5 Score (Standard GEC)	**22.36%**	—	Precision weighted 2× over Recall (conservative edit penalty)
Exact Match (EM) Accuracy	**5.98%**	0.00%	Strict full-sentence character-for-character match

6. Qualitative Validation on Mandatory Test Sentences

The system was validated against canonical diagnostic sentences representing foundational ESL error patterns. In each case, GrammerSense corrected the syntax, localized the offending span, and assigned the verified tier:

#	Input Ungrammatical Sentence	Model Corrected Output	Detected Spans & Category
1	`She go to school yesterday.`	**She went to school yesterday.**	Span [4:6] 'go' → 'went' **DIRECT: Verb Tense / Past**
2	`I have seen him yesterday.`	**I saw him yesterday.**	Span [2:11] 'have seen' → 'saw' **DIRECT: Aspect / Temporal Clashes**
3	`They is playing football.`	**They are playing football.**	Span [5:7] 'is' → 'are' **DIRECT: Subject-Verb Agreement**
4	`He don't know nothing about it.`	**He doesn't know anything about it.**	Span [3:8] 'don't' → 'doesn't', [14:21] 'nothing' → 'anything' **DIRECT: Double Negation / SVA**
5	`She is more taller than me.`	**She is taller than me.**	Span [7:11] 'more' deleted **INFERRED: Comparative Redundancy**
6	`creat brotherhood realtionship .`	**create brotherhood relationships.**	Spans 'creat' → 'create', 'realtionship' → 'relationships' **DIRECT: Typo / Spelling**

7. Comprehensive Test Suite & Quality Assurance (29/29 Passed)

Test Suite Module	Coverage Scope	Tests	Status
`tests/test_api.py`	Healthcheck, /correct, /detect, /candidates, /evaluate, schema validation	**5**	**PASSED**
`tests/test_preprocessing.py`	Span alignment, character offset fidelity, whitespace & casing preservation	**5**	**PASSED**
`tests/test_detection.py`	Rule-based taxonomy verification, SVA, tense, spelling, tier tagging	**10**	**PASSED**
`tests/test_correction.py`	Candidate ranking, confidence scoring, visual diff generation, empty input	**5**	**PASSED**
`tests/test_evaluation.py`	GLEU computation, BLEU computation, Token P/R/F0.5, dataset iteration	**4**	**PASSED**
Total Test Suite	**Full End-to-End System Verification**	**29**	**100% PASSED**

8. User Interface & API Integration

• **Zero-Default Clean State:** On load or refresh, text area is completely empty, 'Try Sample' defaults to *'Select a test sentence...'*, and 'Check Grammar' remains disabled until text is provided. Zero auto-firing.
• **Visual Diff Highlighting:** Red strike-through tags for original deletion spans, emerald tags for insertions.
• **REST API Endpoints:** GET /api/v1/health, POST /api/v1/correct, POST /api/v1/detect, POST /api/v1/candidates, POST /api/v1/evaluate.

9. Operational Limitations & Future Roadmap

• **Contextual Boundary:** GEC is presently scoped to individual sentences; multi-paragraph coreference is not modeled.
• **Low-Frequency Idioms:** Idiomatic expressions without literal grammatical errors may occasionally be normalized.
• **ONNX & WebAssembly:** Future work will compile T5 weights into quantized INT8 ONNX models for pure in-browser client execution.